

* Search (Iterative) - $O(n)$.

Search for a key in LL. Return the position where it is found, if not found return -1.

- create temp node.

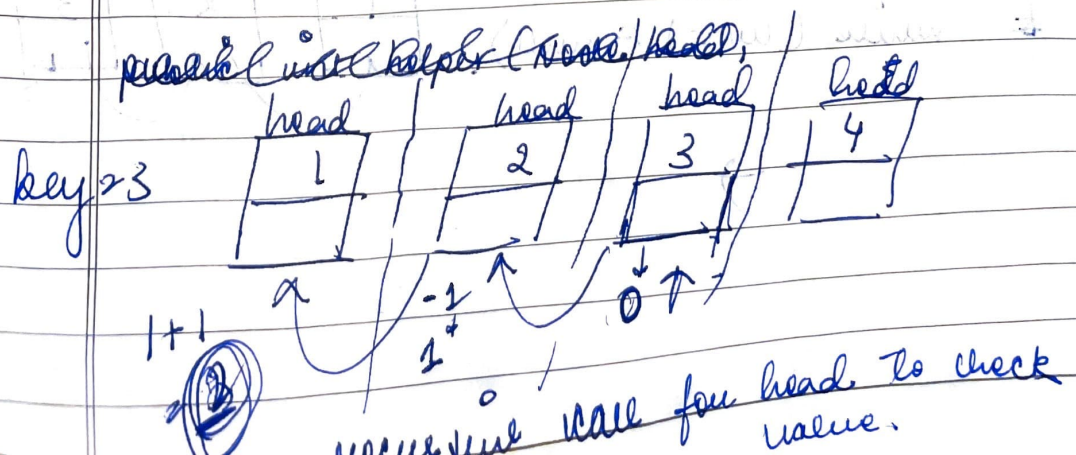
```
public int Search(int key) {
    Node temp = head;
    int i = 0;

    while (temp != null) {
        if (temp.data == key) { // key found
            return i;
            return i;
        }
        temp = temp.next; // updation
        i++;
    }

    // key not found
    return -1;
}
```

* Search (Recursive) $\rightarrow O(n)$.

search using recursion



recursion function

```
public int helper (Node header, int key) {
```

```
    if (head == null) {
```

```
        return -1;
```

```
    }
```

```
    if (head.data == key) {
```

```
        return 0;
```

```
    }
```

```
    int idx = helper (header, key)
```

```
    int idx = helper (header.next, key);
```

```
// header shifts to next node.
```

```
    if (idx == -1) {
```

```
        return -1;
```

```
    } return idx + 1; // when backtracked
```

from key found

```
public int recSearch (int key) {
```

```
    return helper (head, key);
```

```
}
```

Reverse a Linked List (Iteratively)

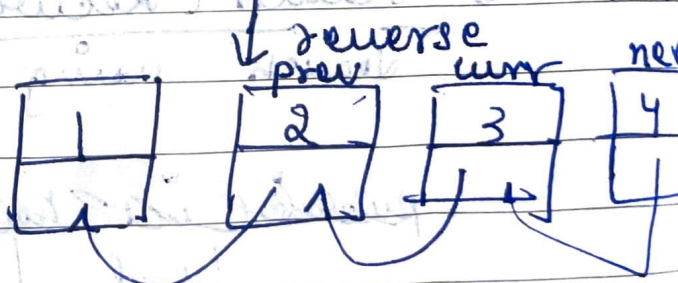
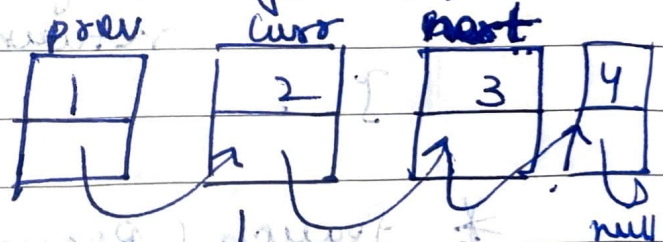
✱

```
next = curr.next
```

```
curr.next = prev
```

```
prev = curr
```

```
curr = next
```



```
while (curr != null)
```

→

```
public void reverse () {
    Node prev = null; // starting node
    Node curr = tail = head;
    Node next;
```

```
    while (curr != null) {
        next = curr.next; // 1
        curr.next = prev; // 2
        prev = curr; // 3, reversal
        curr = next; // 4
    }
```

head = prev; // after entire iteration
last element $\rightarrow$ prev $\rightarrow$ head.

* Find & remove Nth node from End ★★

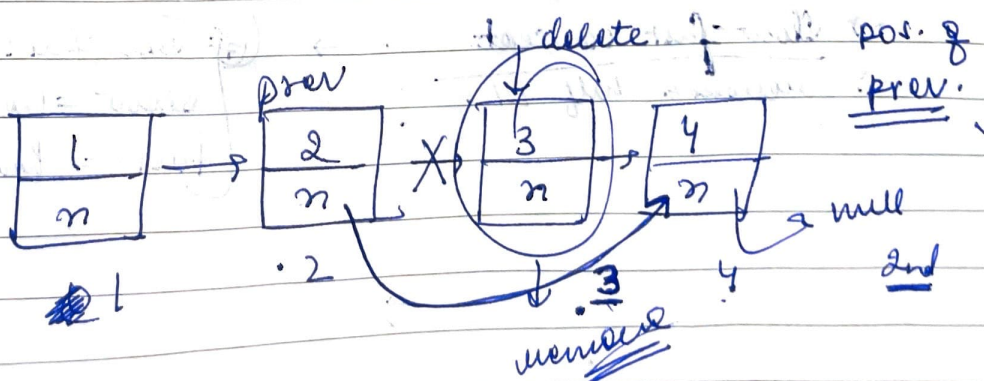
Iterative Approach $\rightarrow O(n)$

$\rightarrow$ To remove nth node moving from end.

then nth end = (size - n + 1) from start

$\uparrow$
position of element to be removed.

$\Rightarrow$ element before / prev this node = (size - n)



```

public void deleteFromNthEnd (int n) {
    // calculate size
    int sz = 0;
    Node temp = head;
    while (temp != null) {
        temp = temp.next;
        sz++;
    }
    if (n == sz)
        head = head.next; // remove from start
    return;

    // sz - n → prev
    int i = 1;
    int iToFind = sz - n;
    Node prev = head;
    while (i < iToFind) {
        prev = prev.next;
        i++;
    }
    prev.next = prev.next.next;
    return;
}

```

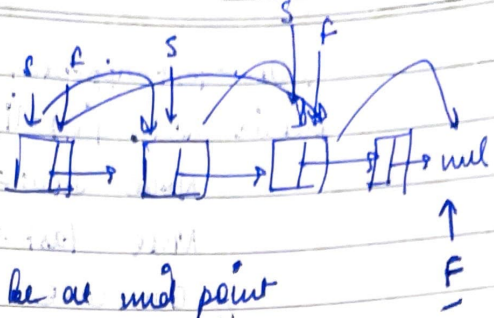
* check if LL is a palindrome

→ slow-fast concept
 → reverse half LL

→ slow fast
 Slow = head // + 1
 fast = head // + 2

① Find mid Node

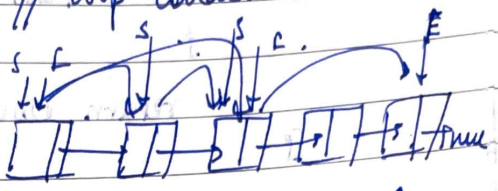
For Even Case



⇒ Slow pointer will be at mid point
Exactly where fast pointer is at null

⇒ fast != null // loop condition

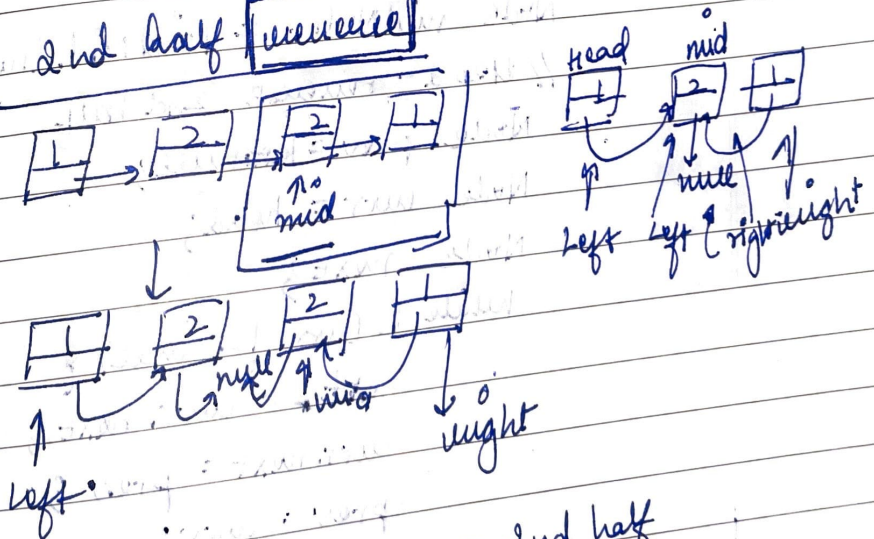
For odd case



⇒ Slow pointer will be at mid point
Exactly where fast pointer is pointing to null

fast.next != null

② 2nd half recursion



③ check if 1st half == 2nd half (left) (right)

// Slow-fast Approach

```

public Node findMid(Node head) {
    Node slow = head;
    Node fast = head;

    // Even case - odd case conditions check
    while (fast != null && fast.next != null) {
        slow = slow.next; // +1
        fast = fast.next.next; // +2
    }

    return slow; // slow -> mid value.
}

```

```

public boolean isPalindrome() {
    if (head == null || head.next == null) {
        // If first or first node is null or doesn't exist
        return true;
    }
}

```

// Step 1 - find mid
~~create mid~~

```

Node midNode = findMid(head);

```

// Step 2 - reverse 2nd half

```

Node prev = null;
Node curr = head;
Node next;
while (curr != null) {
    next = curr.next;
    curr.next = prev;
    prev = curr;
    curr = next;
}

```

// Check left half & right

```

Node right = prev; // head from 2nd half
Node left = head;

```

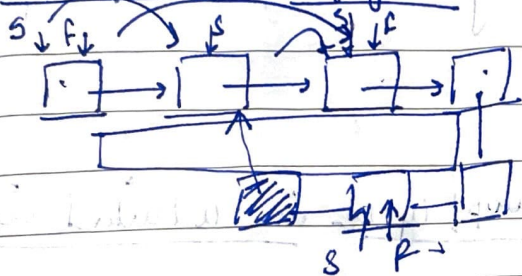
```

    prev = prev.next;
    if (left.data != right.data) {
        return false;
    }
    left = left.next;
    right = right.next;
}
return true;

```

19.2

* To Detect a loop/cycle in a linked list



Floyd's cycle finding Algorithm

slow fast pointers
 $\uparrow$ $\uparrow$
 $t1$ $t2$

when slow = fast
 cycle = true
 else false

	distance
0 moves	→ 0
1 move	→ 1
2 moves	→ 2
3 moves	→ 3

```

public static boolean isCycle() {
    Node slow = head;
    Node fast = head;

```

```

    while (fast != null && fast.next != null) {
        slow = slow.next; // +1
        fast = fast.next.next; // +2
    }

```